1. Introduction

The purpose of this Test Plan is to define the testing strategy, scope, resources, schedule, and deliverables for the Banking Transaction Management System (BTMS) database testing project. Testing will focus exclusively on database-level validation using SQL INSERT and UPDATE operations to ensure:

- Data integrity
- Business rule enforcement
- Referential integrity
- Trigger and constraint validation
- Accurate account balance handling
- Proper transaction and fee processing

The testing activities will validate that the database schema, constraints, triggers, and relationships function according to the Business Requirements Document (BRD).

---

2. Scope of Testing

In Scope

The following modules and database components are included in testing:

Customer Management

- Customer creation
- Customer updates
- Email uniqueness validation
- Phone number validation

Account Management

- Account creation
- Account type validation
- Balance validation
- Multi-account handling

Transaction Management

- Deposits
- Withdrawals
- Transfers

- Bill payments
- Balance updates
- Transaction status validation
- Daily transfer limit checks

Transaction Categories

- Category assignment
- Category integrity validation

Fees and Charges

- Fee creation
- Transfer fee logic
- Failed transaction fee logic
- Fee percentage validation

Branch and Employee Management

- Branch creation
- Employee creation
- Foreign key relationship validation

Database Objects

- Primary keys
- Foreign keys
- CHECK constraints
- ENUM validations
- Triggers
- Auto-increment functionality

Out of Scope

The following items are excluded from testing:

- Front-end UI testing
- API testing
- Reporting dashboards

---

3. Test Objectives

The objectives of BTMS database testing are to:

1. Verify all tables are created successfully with correct schema definitions.
2. Validate INSERT and UPDATE operations across all tables.
3. Ensure primary key and foreign key constraints work correctly.
4. Verify CHECK constraints and ENUM restrictions.
5. Validate trigger functionality for:
   - Balance updates
   - Fee calculations
   - Daily transaction limits
6. Ensure failed transactions do not alter balances.
7. Verify transaction atomicity for transfers.
8. Validate business rules for account balances and fees.
9. Ensure data consistency across related tables.
10. Confirm database response time meets the requirement of ≤ 2 seconds for INSERT/UPDATE operations.

---

4. Testing Approach

Testing Type

- Functional Database Testing
- Constraint Validation Testing
- Data Integrity Testing
- Trigger Testing
- Negative Testing
- Boundary Value Testing

Testing Methodology

Phase 1 – Schema Validation

Validate:

- Table structures
- Data types
- PK/FK relationships
- Auto-increment functionality
- ENUM values

- CHECK constraints

Phase 2 – Positive Testing

Execute valid INSERT and UPDATE queries to verify:

- Correct data insertion
- Proper balance updates
- Valid fee creation
- Successful transaction processing

Examples:

- Insert valid customer
- Insert valid transfer transaction
- Update account balance correctly

Phase 3 – Negative Testing

Execute invalid queries to verify database rejection and error handling.
Examples:

- Duplicate email insertion
- Negative transaction amount
- Invalid ENUM value
- Overdraft withdrawal
- Invalid foreign key references

Phase 4 – Business Rule Validation

Validate:

- Automatic balance updates
- Daily transaction limits
- Fee application logic
- Failed transaction handling
- Transfer atomicity

Phase 5 – Data Integrity Validation

Use SELECT queries to validate:

- Balance accuracy
- Fee totals
- Transaction consistency
- Relationship integrity

Test Design Techniques

- Equivalence Partitioning
- Boundary Value Analysis
- Error Guessing
- Constraint Testing

---

5. Test Schedule

Phase	Activity
Phase 1	Requirement Analysis & Test Planning
Phase 2	Test Case Design
Phase 5	Test Execution
Phase 6	Defect Logging & Retesting
Phase 7	Regression Testing

Total Estimated Duration:

2 Working Man Days

---

6. Test Environment

Database Platforms

- MySQL Workbench

Environment Configuration

Component	Specification
OS	Windows
DB Tool	MySQL Workbench / SQL Server Management Studio
Database Type	Relational Database
SQL Support	INSERT, UPDATE
Test Data	Manual + Scripted

Required Database Objects

- Tables
- Constraints
- Triggers
- Stored test data

---

7. Resources & Responsibilities

Role	Responsibility
QC Lead	Test planning, review, reporting
Database Tester	Execute SQL test cases
Developer	Fix defects and validate triggers
Business Analyst	Clarify business rules

Resource Requirements

- 1 QC Lead
- 2 Database Testers
- 1 Backend Developer

---

8. Risks & Mitigation

Risk	Impact	Mitigation
Incomplete business rules	Incorrect testing coverage	Conduct BR
Trigger logic errors	Incorrect balances/fees	Perform det
Invalid test data	False test results	Prepare val
Environment instability	Testing delays	Maintain ba
Constraint differences between MySQL and SQL Server	Inconsistent behavior	Test separa
Data corruption during testing	Incorrect outcomes	Use isolated

---

9. Test Deliverables

The following deliverables will be produced during the testing lifecycle:

Test Planning Documents

- Test Plan

Test Design Documents

- Test Cases

Execution Deliverables

- Test Execution Report
- Defect Reports
- Retest Results

Final Deliverables

- Test Summary Report
- Defect Closure Report

---

10. Entry & Exit Criteria

Entry Criteria

Testing will begin when:

- BRD is approved
- Database schema is deployed
- Tables and constraints are created
- Triggers are implemented
- Test environment is available
- Test data is prepared

Exit Criteria

Testing will be completed when:

- All planned test cases are executed
- Critical and high-severity defects are fixed
- No open blocker defects remain
- Business rules are validated successfully
- Data integrity is confirmed
- Test summary report is approved
- Stakeholder sign-off is received

Conclusion

This Test Plan ensures comprehensive database validation for the Banking Transaction Management System (BTMS). The testing process focuses on validating business-critical database operations, maintaining transactional integrity, enforcing constraints, and ensuring reliable balance and fee management through SQL-based testing.